

Diseñando para responder, no para esperar

G. D. Monzón Flores
7690-20-20745 Universidad Mariano Gálvez
Seminario de Tecnología de Información
gmonzonf1@miumg.edu.gt

Resumen La programación reactiva se ha consolidado como una de las soluciones más sólidas y eficientes para resolver los desafíos de la arquitectura de software en esta era, la cual está fuertemente marcada por la demanda de alta concurrencia, el procesamiento asíncrono de las peticiones y los flujos de datos en tiempo real. En el ecosistema moderno, muchas aplicaciones en sectores críticos, como pueden ser la banca o el comercio electrónico masivo, se enfrentan diariamente al reto de procesar miles y miles de solicitudes de manera simultánea, sin afectar la experiencia del usuario ni agotar los recursos dedicados a la infraestructura.

Bajo este contexto, este ensayo pretende examinar de manera exhaustiva las buenas prácticas al momento de diseñar y desarrollar aplicaciones reactivas. A lo largo de este documento, se analizan los fundamentos del Manifiesto Reactivo y se profundiza en las estrategias clave para evitar operaciones bloqueantes, estructurar flujos legibles y asegurar la resiliencia del software mediante pruebas rigurosas.

El objetivo es demostrar que la reactividad no es simplemente un conjunto de herramientas técnicas, sino un enfoque arquitectónico que transforma la tolerancia a fallos y la escalabilidad del software actual.

Palabras claves: programación reactiva, aplicaciones reactivas, escalabilidad, asincronía, concurrencia.

Desarrollo del tema Actualmente, muchas aplicaciones deben responder a una gran cantidad de solicitudes al mismo tiempo. Una plataforma de ventas, una aplicación bancaria o un servicio de mensajería puede recibir peticiones de cientos o incluso miles de usuarios de manera simultánea. Cada solicitud puede necesitar consultar en la base de datos, comunicarse con otro servicio o esperar la respuesta de un servicio externo. Si estas operaciones no son controladas correctamente, el sistema puede volverse lento, consumir muchísimos recursos o dejar de funcionar.

Las aplicaciones reactivas buscan atender este problema mediante un modelo basado en eventos y operaciones no bloqueantes. En un modelo tradicional, un proceso puede permanecer esperando que una operación termine. Durante ese tiempo, el proceso bloquea el poder ejecutar otros procesos. Por el lado contrario, en una aplicación reactiva el sistema puede continuar atendiendo otros eventos y procesar la respuesta cuando se encuentre disponible.

Según Bonér et al. (2014), un sistema reactivo debe ser responsivo, resiliente, elástico y orientado a mensajes. Ser responsivo significa que el sistema debe responder en un tiempo razonable. La resiliencia permite que continúe funcionando cuando ocurre un error, mientras que la elasticidad se relaciona con su capacidad de adaptarse a los cambios en la carga de trabajo. La comunicación mediante mensajes también ayuda a reducir la dependencia directa entre los componentes.

Sin embargo, desarrollar una aplicación reactiva no consiste únicamente en instalar una librería o utilizar un framework que incluya funciones asíncronas. La reactividad debe mantenerse durante todo

el flujo de procesamiento. Si una parte de la aplicación utiliza operaciones no bloqueantes, pero otra realiza una consulta tradicional que detiene el hilo, el beneficio puede reducirse considerablemente. La aplicación tendría una apariencia reactiva, pero conservaría el mismo problema en una parte importante de su ejecución.

Una de las principales buenas prácticas consiste en evitar las operaciones bloqueantes. Estas suelen aparecer al consultar bases de datos, leer archivos o comunicarse con servicios externos. El problema no se encuentra en que una operación necesite tiempo para responder, sino en mantener ocupado un recurso mientras se espera el resultado. Cuando esto ocurre con muchos usuarios, el servidor necesita mantener una gran cantidad de hilos, lo que aumenta el consumo de memoria y puede afectar el rendimiento general del sistema.

Para evitar este problema se deben utilizar controladores, clientes y herramientas compatibles con operaciones no bloqueantes. No tendría sentido utilizar un framework reactivo en la capa encargada de recibir solicitudes si el acceso a la base de datos continúa utilizando un controlador completamente bloqueante. Para que este modelo funcione correctamente, todas las partes importantes del flujo deben mantener un comportamiento compatible con la programación reactiva.

Otra buena práctica es mantener flujos de procesamiento simples y legibles. La programación reactiva incluye operadores para transformar, filtrar, combinar y controlar eventos. Estas herramientas permiten construir procesos complejos, pero también pueden producir código difícil de comprender cuando se utilizan sin una estructura clara. Encadenar demasiadas operaciones en un mismo bloque dificulta identificar dónde ocurre un error o cuál es la responsabilidad de cada parte.

Por esta razón, es recomendable dividir los flujos en funciones pequeñas y utilizar nombres que expliquen lo que realiza cada operación. También debe evitarse colocar toda la lógica de negocio dentro de los controladores. El controlador debería limitarse a recibir la solicitud y enviar la respuesta, mientras que las reglas del sistema deben mantenerse en servicios o componentes separados. Esta división facilita las pruebas, el mantenimiento y la reutilización del código.

El manejo de errores representa otro punto a resaltar. En un flujo reactivo, una excepción puede detener la secuencia completa si no se controla correctamente. La aplicación debe diferenciar entre errores esperados y errores inesperados. Por ejemplo, que un usuario no exista puede ser una condición prevista y debe producir una respuesta controlada. En cambio, una pérdida de conexión con la base de datos puede requerir un registro más detallado, un intento de recuperación o una respuesta temporal indicando que el servicio no se encuentra disponible.

En algunos casos pueden aplicarse reintentos automáticos, pero estos deben utilizarse con cuidado. Repetir una operación sin establecer límites puede empeorar el problema. Si un servicio externo se encuentra caído y todas las solicitudes se repiten constantemente, la carga puede aumentar y retrasar su recuperación. Los reintentos deben tener una cantidad máxima, un tiempo de espera y utilizarse únicamente en operaciones que puedan repetirse de forma segura.

Otra práctica necesaria es controlar la velocidad con la que se producen y consumen los datos. En un flujo reactivo, una fuente puede generar información más rápido de lo que otro componente puede procesarla. Este problema se relaciona con el concepto de contrapresión o *backpressure*. Reactive Streams (2015) propone mecanismos para que el consumidor indique cuántos elementos puede procesar, evitando que el productor envíe información de manera ilimitada. Dependiendo del caso, los datos pueden almacenarse temporalmente, procesarse por grupos, limitarse o descartarse cuando

ya no sean necesarios.

Las pruebas y el monitoreo también deben adaptarse al comportamiento reactivo. No es suficiente comprobar que un método devuelve el resultado esperado. Es necesario evaluar qué sucede cuando un servicio responde lentamente, cuando una conexión se interrumpe o cuando muchas solicitudes llegan al mismo tiempo. Las pruebas de carga permiten medir el tiempo de respuesta, el consumo de memoria, el uso del procesador y la cantidad de errores. Según Kuhn, Hanafée y Allen (2017), la resiliencia debe formar parte de la arquitectura y no depender únicamente de soluciones aplicadas después de que aparece un problema.

Finalmente, una de las prácticas más importantes consiste en determinar si el proyecto realmente necesita programación reactiva. Este paradigma resulta útil en sistemas con muchas operaciones de entrada y salida, conexiones de larga duración o una cantidad elevada de usuarios concurrentes. Sin embargo, una aplicación administrativa pequeña puede funcionar correctamente con una arquitectura tradicional. La decisión debe basarse en los requerimientos, el tipo de operaciones y la experiencia del equipo. La mejor arquitectura no siempre es la más moderna, sino aquella que resuelve el problema de forma clara y eficiente.

Observaciones y comentarios

Durante el análisis se observó que muchas de las dificultades relacionadas con la programación reactiva aparecen cuando este paradigma se adopta sin comprender realmente sus principios. Utilizar herramientas reactivas en una sola parte de la aplicación no garantiza un mejor rendimiento si el resto del flujo continúa utilizando operaciones bloqueantes. También se identificó que la complejidad del código puede aumentar cuando los procesos no se organizan de forma clara o se encadenan demasiadas operaciones en un mismo bloque. Por esta razón, la experiencia del equipo y la claridad del diseño resultan tan importantes como la tecnología seleccionada. Una aplicación reactiva debería facilitar la respuesta ante una carga elevada y no convertirse en una complicación innecesaria para el desarrollo y mantenimiento del sistema.

Conclusiones

1. Las aplicaciones reactivas pueden aprovechar mejor los recursos cuando deben atender muchas operaciones de entrada y salida de forma simultánea.
2. Evitar operaciones bloqueantes es necesario para mantener los beneficios del modelo reactivo durante todo el flujo de la aplicación.
3. El manejo de errores, la contrapresión, las pruebas de carga y el monitoreo deben considerarse desde el diseño y no únicamente al finalizar el desarrollo.
4. La programación reactiva no representa una solución adecuada para todos los proyectos, debido a que puede aumentar la complejidad sin aportar una mejora necesaria.
5. El éxito de una aplicación reactiva depende más de las decisiones de arquitectura y desarrollo que del framework utilizado.

Bibliografía

Bonér, J., Farley, D., Kuhn, R., y Thompson, M. (2014). *The Reactive Manifesto*. <https://www.reactivemanifesto.org/>

Kuhn, R., Hanafée, B., y Allen, J. (2017). *Reactive design patterns*. Manning Publications.

Reactive Streams. (2015). *Reactive Streams*. <https://www.reactive-streams.org/>

Repositorio

<https://github.com/DaniSaurioM/ForosAcademicos-Seminario>